Finite Automata in Haskell

V. Velkey, W. Vromen

Friday 2nd June, 2023

Abstract

Finite state automata are popular tools to analyse formal languages. This project aims to implement finite state automata, both deterministic and nondeterministic. We also implement regular expressions and generate words from their language. Finally, we implement translations between our objects and run several tests to verify our implementation.

Contents

1 Introduction

Finite state automata are mathematical models of computation. They are mostly used to study regular languages, i.e. languages with a certain structure. This structure is also captured by regular expressions.

In this report we discuss our Haskell-implementation of finite state automata and regular expressions. In particular, we implemented deterministic finite state automata, non-deterministic finite state automata and regular expressions. The goal is to be able to compare the languages generated by regular expressions and the languages accepted by the various automata.

In the first section we will provide some formal background for automata theory and regular languages. For the second part we will discuss our implementation. Afterwards in section three we will show some cool stuff / provable stuff that one can do with the program, after which there will be an epic conclusion in the final section.

2 Background Theory

Automata are mathematical models of computation. They are used in several areas of computer science and linguistics. Finite state automata, one of the most basic models, are used in, among other things, text processing and compilers. Here we will give a brief overview of the theory, which should be enough to make one understand the report. For a more detailed exposition, see for example [Sip13].

2.1 Formal Languages

Automata are concerned with *formal languages*. A language in our setting is nothing more than a set of strings, or *words*, constructed from a given (finite) *alphabet*. An alphabet is typically denoted by the Greek capital Σ and consists of *symbols* or characters. A string over an alphabet is nothing more than a finite sequence of symbols. If A is a set, then the set of all finite sequences over A is denoted A^*. Hence A^* is the full language over alphabet A.

One special symbol that is a member of A^* for any A is ϵ. This character denotes the *empty string* and is just the word of length 0.

An example of an alphabet is $\Sigma = \{0, 1\}$ and one of the many possible languages over Σ is $B = \{0w \mid w \in \Sigma^*\}$. This language contains all binary strings starting with 0 and nothing more.

2.2 Combining Languages and Regular Expressions

We define several operations over languages to construct new ones. These are called the *regular operations*.

Let A and B be formal languages over alphabet Σ. Then:

- Their union, denoted as $A \cup B$, is defined as $\{w \mid w \in A \cup B\}$.
- Their concatenation, denoted as $A \circ B$ is defined as $\{wv \mid w \in A,\ v \in B\}$.
- We have already seen the finite sequence operator *, called the *Kleene star*, to get $A^* := \{w_1 w_2 \dots w_n \mid \forall i : w_i \in A,\ n \in \mathbb{N}\}$.
- Finally, we also have $A^+ := A^* \setminus \{\epsilon\}$ to get all non-empty sequences made with words from A.

These operations naturally give rise to *regular expressions*. A regular expression r always corresponds to a language, $\mathcal{L}(r)$.

Definition 2.1 (Regular Expressions)**.** Let Σ be an alphabet. The set $\mathcal{R}$ of regular expressions over Σ are recursively defined as follows:

- $\emptyset \in \mathcal{R}$, denoting the empty language: $\mathcal{L}(\emptyset) = \emptyset$.
- $\epsilon \in \mathcal{R}$, denoting $\mathcal{L}(\epsilon) = \{\epsilon\}$.
- $a \in \mathcal{R}$, with $\mathcal{L}(a) = \{a\}$ for all $a \in \Sigma$.

If $r, r' \in \mathcal{R}$, then:

- $r + r' \in R$, also written as $r|r'$, corresponding to $\mathcal{L}(r) \cup \mathcal{L}(r')$.
- $rr' \in \mathcal{R}$ with $\mathcal{L}(rr') = \mathcal{L}(r) \circ \mathcal{L}(r')$.
- r^* corresponding to $\mathcal{L}(r)^*$.
- r^+ with $\mathcal{L}(r^+)^+ = \mathcal{L}(r)^+$.

A regular expression r is like a pattern; any word that matches the pattern is *generated by* r. Languages generated by regular expressions are called *regular languages*.

2.3 Finite State Automata

As stated earlier, a finite state automaton (FSA) is a model of computation. Specifically, an FSA operates on words. When inputting a word into an FSA, it will either accept or reject. The set of all words that are accepted by an FSA A is called the language (recognized) by A. It is denoted $\mathcal{L}(A)$. We distinguish deterministic finite state automata (DFAs) from non-deterministic finite state automata ((ϵ-)NFAs). The formal definition of a DFA is as follows:

Definition 2.2. A DFA D is a 5-tuple $(S, \Sigma, \delta, s_0, F)$, where:

- S is a non-empty set of *states*.
- Σ is the alphabet on which D operates.
- $\delta : S \times \Sigma \to S$ is the transition function[1]
- $s_0 \in S$ is the starting state.
- $F \subseteq S$ is the set of accepting states.

The computation of a word w on DFA $D = (S, \Sigma, \delta, s_0, F)$ will goes as follows:

1. Let the current state s be s_0.
2. - If $w = \epsilon$, accept if $s \in F$ and reject otherwise.
   - If $w = aw'$ where $a \in \Sigma$, let s be the next state $\delta(s, a)$ and let $w = w'$. Repeat step 2.

We have two remarks about this procedure:

- It is up for debate if a DFA rejects or fails when it reads a symbol that is not in the alphabet. Our implementation will reject for a more user-friendly experience.
- An NFA could end up in multiple states when reading a symbol. The idea is that the machine will non-deterministically choose to which state to go. If one possible path of computation ends up in an accepting state, the machine will accept. Note that some paths may end due to the machine reading a symbol a in a state s such that there is no t with $(s, a, t) \in \Delta$. In that case this path will count as not ending up in an accepting state.

Two FSAs are called *equivalent* just in case they recognize the same language. It may not be surprising that for every DFA there is an equivalent NFA: every function is a relation after all. However it turns out that for every NFA there is also an equivalent DFA. More on this in the section on the power set construction. ADD REF TO PAGE. Moreover, every language that is accepted by an FSA is also generated by some regular expression and vice versa. I.e. the languages recognized by FSAs are exactly the regular ones. In the next sections we will discuss how the Haskell program shows these facts.

3 DFA Implementation

This describes our implementation of Deterministic Finite State Automata. As in the definition, a DFA consists of a tuple $(Q, \Sigma, \delta, q_{start}, A)$ representing the set of states, the alphabet, the transition function, the start state and the accept states.

[1]When considering an NFA, the only difference is that this is a transition relation $\Delta \subseteq S \times \Sigma \times S$.

```haskell
{-# LANGUAGE FlexibleInstances #-}
module DFA where

import Data.List
import Test.QuickCheck

type Symbol = Char
data DFA a = DFA { states :: [a]
                 , alphabet:: [Symbol]
                 , delta :: Symbol -> a -> a
                 , start:: a
                 , acceptstate:: [a]}
instance Show a => Show (DFA a) where
    show (DFA sts alph del strt acc)
      = "DFA" ++ "("++ show sts ++ "," ++ show alph ++ "," ++ show1 del ++ "," ++ show strt
          ++ "," ++ show acc ++")" where
      show1 f = show ["d" ++ "(" ++ show sym ++ "," ++ show st ++ ")" ++ " = " ++ show (f
          sym st) | sym <- alph, st <- sts ]
```

We implement a basic evaluation function that upon input from the string, evaluates if the string is in the language.

```haskell
evaluate:: (Eq a, Ord a) => DFA a -> String -> Bool
evaluate df st = all (`elem` alphabet df) st && -- captures that all element of string in
    alphabet
                 stateArr (start df) st `elem` acceptstate df where
                     stateArr  q [] = q
                     stateArr q (x:xs) = stateArr (delta df x q) xs
```

We write a toy example DFA on the alphabet $\Sigma = \{0, 1\}$ computing the language of words starting with a 0.

```haskell
zeroStart:: DFA Int
zeroStart = DFA [0,1,2] ['0', '1'] deltazero 0 [1] where
    deltazero:: Symbol -> Int -> Int
    deltazero char st
     | st == 0 && char == '0' = 1
     | st == 0 && char == '1' = 2
     | st == 1 = 1
     | st == 2 = 2
     | otherwise = -1
```

Some transforms of a DFA.

```haskell
flipDFA :: (Eq a, Ord a) => DFA a -> DFA a
flipDFA (DFA sts alp del st acc) = DFA sts alp del st (sts \\ acc)

reachables :: Eq a => DFA a -> [a]
reachables dfa = reachableInSteps [start dfa] where
  reachableInSteps ts | all (`elem` ts) (concatMap allSuccessors ts) = ts
                      | otherwise = reachableInSteps (nub $ ts ++ concatMap allSuccessors ts
                          )
  allSuccessors st = nub (st : [delta dfa sym st | sym <- alphabet dfa])


cutDFA :: Eq a => DFA a -> DFA a
cutDFA d = DFA sts alp delt strt acc where
  sts = reachables d
  alp = alphabet d
  strt = start d
  acc = reachables d `intersect` acceptstate d
  delt = delta d
```

We make DFA-s instance of Arbitrary as follows. We use solution to Homework 2.

```haskell
-- recursively make a valuation function for these worlds:
```

```
randomFunFromTo :: (Eq a, Ord a, Arbitrary a) => [a] -> [a] -> Gen (a -> a)
randomFunFromTo [] _ = return (const undefined)
randomFunFromTo (w:ws) ps = do
    f <- randomFunFromTo ws ps
    wResult <- elements ps
    return $ \v -> if v == w then wResult else f v

randomDelta :: (Eq a, Ord a, Arbitrary a) => [Symbol] -> [a] -> [a] -> Gen (Symbol -> a ->
    a)
randomDelta [] _ _ = return (const undefined)
randomDelta (sym:syms) ds ps = do
    f <- randomDelta syms ds ps
    wResult <- randomFunFromTo ds ps
    return $ \sy -> if sy == sym then wResult else f sy

instance Arbitrary (DFA Int) where
    arbitrary = do
        -- choose a set of up to 4 worlds:
        sts <- (0 :) <$> sublistOf [1..3]
        let sym = ['0', '1']
        delt <- randomDelta sym sts sts
        strt <- elements sts
        accraw <-  sublistOf sts
        let acc = nub (0:accraw)
        return $ DFA sts sym delt strt acc


languageIsEmpty :: Eq a => DFA a -> Bool
languageIsEmpty d = null $ reachables d `intersect` acceptstate d
```

4 NFA Implementation

This describes our implementation of Nondeterministic Finite State Automata. As in the definition, an NFA consists of a tuple $(Q, \Sigma, \delta, q_{start}, A)$ representing the set of states, the alphabet, the transition function, the start state and the accept states.

```
{-# LANGUAGE FlexibleInstances #-}
module NFA where

import Data.List
import DFA
import Test.QuickCheck


data NFA a= NFA { statesNF :: [a]
                , alphabetNF:: [Symbol]
                , deltaNF :: Symbol -> a -> [a]
                , startNF:: a
                , acceptstateNF:: [a]}

instance Show a => Show (NFA a) where
    show (NFA sts alph del strt acc)
      = "NFA" ++ "("++ show sts ++ "," ++ show alph ++ "," ++ show1 del ++ "," ++ show strt
           ++ "," ++ show acc ++")" where
      show1 f = show ["d" ++ "(" ++ show sym ++ "," ++ show st ++ ")" ++ " = " ++ show (f
          sym st) | sym <- alph, st <- sts ]
```

We also implement a similar, but more complicated transition function suitable for NFA-s.

```
evaluateNF:: Eq a => NFA a-> String -> Bool
evaluateNF nf st = all (`elem` alphabetNF nf) st && -- captures elements of string in
    alphabet
             any (`elem` acceptstateNF nf) (stateArrNF' [startNF nf] st)  where
                stateArrNF'  qs [] = qs
                stateArrNF'  [] _ = []
```

```
    stateArrNF'  [q] (x:xs) = stateArrNF' (deltaNF nf x q) xs --recursion on
        string
    stateArrNF'  (q:qs) (x:xs) = stateArrNF' [q] (x : xs) `union` stateArrNF'
        qs (x : xs) --recursion on statespace
```

We make NFA -s instance of Arbitrary by slightly modifying the relevant code for DFA-s.

```
-- recursively make a valuation function for these worlds:
randomFunFromTolist :: (Eq a, Arbitrary a) => [a] -> [a] -> Gen (a -> [a])
randomFunFromTolist [] _ = return (const undefined)
randomFunFromTolist (w:ws) ps = do
    f <- randomFunFromTolist ws ps
    wResult <- sublistOf ps
    return $ \v -> if v == w then wResult else f v

randomDeltaNF :: (Eq a, Arbitrary a) => [Symbol] -> [a] -> [a] -> Gen (Symbol -> a -> [a])
randomDeltaNF [] _ _ = return (const undefined)
randomDeltaNF (sym:syms) ds ps = do
    f <- randomDeltaNF syms ds ps
    wResult <- randomFunFromTolist ds ps
    return $ \sy -> if sy == sym then wResult else f sy

instance Arbitrary (NFA Int) where
    arbitrary = do
        -- choose a set of up to 10 worlds:
        sts <- (0 :) <$> sublistOf [1..5]
        let sym = ['0', '1']
        delt <- randomDeltaNF sym sts sts
        strt <- elements sts
        accraw <- sublistOf sts
        let acc = nub (0:accraw)
        return $ NFA sts sym delt strt acc
```

5 Implementation of ϵ-NFA-s

Here we modify our implementation of NFA-s to account for epsilon transitions. This requires some technical apparatus. First we give the type structure and define some helper functions to calculate ϵ-closure of states. For this we use the blog answer [Jel14].

```
{-# LANGUAGE FlexibleInstances #-}
module ENFA where

import Data.List
import DFA
import NFA
import Test.QuickCheck
data ENFA a = ENFA { statesENF :: [a]
                   , alphabetENF:: [Symbol]
                   , deltaENF :: Symbol -> a -> [a]
                   , epTrans:: [(a, a)]
                   , startENF:: a
                   , acceptstateENF:: [a]}

instance Show a => Show (ENFA a) where
    show (ENFA sts alph del eps strt acc)
      = "ENFA" ++ "("++ show sts ++ "," ++ show alph ++ "," ++ show1 del ++ "," ++ show eps
          ++ "," ++ show strt ++ "," ++ show acc ++")" where
      show1 f = show ["d" ++ "(" ++ show sym ++ "," ++ show st ++ ")" ++ " = " ++ show (f
          sym st) | sym <- alph, st <- sts ]
-- Taken from https://stackoverflow.com/questions/19212558/transitive-closure-from-a-list-
    using-haskell
trClose :: Eq a => [(a, a)] -> [(a, a)]
trClose closure
  | closure == closureUntilNow = closure
  | otherwise                  = trClose closureUntilNow
```

```
  where closureUntilNow =
          nub $ closure ++ [(a, c) | (a, b) <- closure, (b', c) <- closure, b == b']


refClose:: Eq a => [a] -> [(a,a)] -> [(a,a)]
refClose as ps = nub (ps ++ [(x,x) | x <- as])

rtClose:: (Eq a, Ord a) => ENFA a -> [a] -> [a]
rtClose nf qs = sort (unionL [[p | p <- statesENF nf , (q, p) `elem` trClose rcl ] | q<- qs
    ] ) where
                                            rcl = refClose (statesENF nf) (epTrans nf)
```

Finally, we modify the the evaluation function.

```
unionL:: Eq a => [[a]] -> [a]
unionL = foldr union []

evaluateENF:: (Eq a, Ord a) => ENFA a -> String -> Bool
evaluateENF nf st = all (`elem` alphabetENF nf) st && -- captures elements of string in
    alphabet
              any (`elem` acceptstateENF nf) (stateArrENF' [startENF nf] st)  where
                 stateArrENF' qs [] = rtClose nf qs
                 stateArrENF' [] _ = []
                 stateArrENF' [q] (x:xs)  = rtClose nf (stateArrENF' (unionL (map (deltaENF
                     nf x) (rtClose nf [q]))) xs )
                 stateArrENF' (q:qs) (x:xs) = stateArrENF' [q] (x : xs) `union` stateArrENF'
                      qs (x : xs) --recursion on statespace
```

We make ENFA -s instance of Arbitrary by slightly modifying the relevant code for NFA-s.

```


instance Arbitrary (ENFA Int) where
    arbitrary = do
        -- choose a set of up to 10 worlds:
        sts <- (0 :) <$> sublistOf [1..5]
        let sym = ['0', '1']
        delt <- randomDeltaNF sym sts sts
        eps <- sublistOf [(x, y) | x <- sts, y <- sts]
        strt <- elements sts
        accraw <- sublistOf sts
        let acc = nub (0:accraw)
        return $ ENFA sts sym delt eps strt acc
```

6 Regular expressions

Here we implement regular expressions and a toolset to generate words that the regexp accepts.

```
data RegExp = Empty | Epsilon | R [Symbol] | Union RegExp RegExp | Star RegExp | Con RegExp
     RegExp | Plus RegExp
  deriving (Show, Eq)


regExpUnion :: [RegExp] -> RegExp
regExpUnion = simplify . foldr Union Empty


pRegExp :: RegExp -> String
pRegExp Empty = ""
pRegExp Epsilon = "R e"
pRegExp (R []) = "R e"
pRegExp (R xs) = show xs
pRegExp (Union r1 r2) = "(" ++ pRegExp r1 ++ "|" ++ pRegExp r2 ++ ")"
pRegExp (Star r) = "(" ++  pRegExp r ++ ")*"
pRegExp (Con r1 r2) = pRegExp r1 ++ pRegExp r2
```

```
pRegExp (Plus r) = "(" ++ pRegExp r ++ ")+"

ppRegExp :: RegExp -> IO ()
ppRegExp = print . pRegExp
```

We create an Arbitrary instance for RegExp to be able to generate regular expressions. Note we exclude the Empty expression from the generation, as we cannot generate words from that.

```
instance Arbitrary RegExp where
  arbitrary = sized randomReg where
    randomReg :: Int -> Gen RegExp
    randomReg 0 = R <$> elements ["0", "1"]
    randomReg n = oneof [ R <$> elements ["0", "1"]
                        , Star <$> randomReg (n `div` 8)
                        , Plus <$> randomReg (n `div` 8)
                        , Con <$> randomReg (n `div` 8)
                        <*> randomReg (n `div` 8)
                        , Union <$> randomReg (n `div` 8)
                        <*> randomReg (n `div` 8)
                        , return Epsilon ]
```

```
generateString :: RegExp -> Gen String
generateString = (`generateString'` 5) . simplify
  where generateString' :: RegExp -> Int -> Gen String
        generateString' Empty _ = error "cannot generate from empty language"
        generateString' Epsilon _ = return ""
        generateString' (R xs) _ = return xs
        generateString' (Union r1 r2) n = oneof [generateString' r1 n, generateString' r2 n
            ]
        generateString' (Con r1 r2) n = do
            w <- generateString' r1 n
            v <- generateString' r2 n
            return (w ++ v)
        generateString' (Star _) 0 = return ""
        generateString' (Star r) n = oneof [generateString' Epsilon n, generateString' (Con
             r (Star r)) (n - 1)]
        generateString' (Plus r) n = generateString' (Con r (Star r)) n



simplify :: RegExp -> RegExp
simplify r | r == simplify' r = r
           | otherwise = simplify $ simplify' r
  where
  simplify' :: RegExp -> RegExp
  simplify' Empty = Empty
  simplify' Epsilon = Epsilon
  simplify' (R xs) = R xs
  simplify' (Con Empty Empty) = Empty
  simplify' (Con Epsilon Epsilon) = Epsilon
  simplify' (Con _ Empty) = Empty
  simplify' (Con Empty _) = Empty
  simplify' (Con r' Epsilon) = simplify' r'
  simplify' (Con Epsilon r') = simplify' r'
  simplify' (Con r1 r2) = Con (simplify' r1) (simplify' r2)
  simplify' (Union Empty Empty) = Empty
  simplify' (Union Empty r') = simplify' r'
  simplify' (Union r' Empty) = simplify' r'
  simplify' (Union Epsilon Epsilon) = Epsilon
  simplify' (Union r1 r2) | r1 /= r2 = Union (simplify' r1) (simplify' r2)
                          | otherwise = simplify' r1
  simplify' (Plus Epsilon) = Epsilon
  simplify' (Star Epsilon) = Epsilon
  simplify' (Plus Empty) = Empty
  simplify' (Star Empty) = Epsilon
  simplify' (Star r') = Star (simplify' r')
  simplify' (Plus r') = Plus (simplify' r')
```

7 Transition functions

Here we implement several translation functions between objects. First the powerset construction between NFA-s and DFA-s.

```
module Translation where
import Data.List
import NFA
import DFA
import ENFA
import RegExp
import Data.Maybe (fromJust)



nfaToDFA:: (Eq a, Ord a) => NFA a -> DFA [a]
nfaToDFA (NFA sts alph del strt ac) =
  cutDFA $ DFA sortedStates alph del' (sort [strt]) [st | st <- sortedStates,  intersect st
       ac /= []] where
    del' sy ls = unionL [del sy l | l <- ls]
    sortedStates = map sort $ subsequences sts
```

We extend the powerset construction for ϵ-NFA-s.

```
enfaToDFA:: (Eq a, Ord a) => ENFA a -> DFA [a]
enfaToDFA (ENFA sts alph del eps strt ac) = let nf = ENFA sts alph del eps strt ac in
  cutDFA $ DFA sortedStates alph del' (rtClose nf (sort [strt])) [st | st <- sortedStates,
       intersect st ac /= []] where
    del' sy ls = let nf = ENFA sts alph del eps strt ac in rtClose nf ( unionL [del sy l |
        l <- ls] )
    sortedStates = map sort $ subsequences sts
```

Now we implement transition from RegExp to ENFA.

```
-- RegExp to ENFA
regExpToENFA :: RegExp -> ENFA Int
regExpToENFA Empty   = ENFA [1] [] (\_ _ -> []) [] 1 []
regExpToENFA Epsilon = ENFA [1] [] (\_ _ -> []) [] 1 [1]
regExpToENFA (R [])  = ENFA [1] [] (\_ _ -> []) [] 1 [1]
regExpToENFA (R xs)  = ENFA [0..length xs] (nub xs) delta2 [] 0 [length xs] where
  delta2 symbol state | state >= length xs || state < 0 = []
                      | xs !! state == symbol = [state + 1]
                      | otherwise = []
regExpToENFA (Union r1 r2) = regExpToENFA r1 `unionENFA` makeDisjoint (regExpToENFA r1) (
    regExpToENFA r2)
regExpToENFA (Star r) = starENFA (regExpToENFA r)
regExpToENFA (Con r1 r2) = regExpToENFA r1 `concatENFA` makeDisjoint (regExpToENFA r1) (
    regExpToENFA r2)
regExpToENFA (Plus r) = regExpToENFA r `concatENFA` makeDisjoint (regExpToENFA r) (starENFA
     (regExpToENFA r))


-- function that takes two ENFAs and outputs a relabeling of the second ENFA such that the
    states of both become disjoint
makeDisjoint :: ENFA Int -> ENFA Int -> ENFA Int
makeDisjoint n1 n2 = ENFA states' alphabet' delta' epT start' accept where
  add | minimum (statesENF n2) <= 0 = negate (minimum (statesENF n2)) + (maximum (statesENF
       n1 ++ statesENF n2) + 1)
      | otherwise = maximum (statesENF n1 ++ statesENF n2) + 1
  states' = map (+ add) (statesENF n2)
  alphabet' = alphabetENF n2
  delta' sym state = map (+ add) (deltaENF n2 sym (state - add))
  epT = [(s + add, t + add) | (s,t) <- epTrans n2 ]
  start' = startENF n2 + add
  accept = map (+ add) (acceptstateENF n2)


unionENFA :: ENFA Int -> ENFA Int -> ENFA Int-- Use only if states are disjoint
```

```
unionENFA n1 n2 = ENFA states'' alphabet2 delta'' epT start'' accept where
  states'' = start'' : statesENF n1 ++ statesENF n2
  alphabet2 = alphabetENF n1 `union` alphabetENF n2
  delta'' sym st = deltaENF n1 sym st ++ deltaENF n2 sym st
  epT = epTrans n1 ++ epTrans n2 ++ [(start'', startENF n1), (start'', startENF n2)]
  start'' = maximum (statesENF n2 ++ statesENF n1) + 1
  accept = acceptstateENF n1 ++ acceptstateENF n2

starENFA :: ENFA Int -> ENFA Int
starENFA n = ENFA (statesENF n) (alphabetENF n) (deltaENF n) ep (startENF n) (
    acceptstateENF n)  where
  ep = epTrans n ++ [(startENF n, s)| s <- acceptstateENF n] ++ [(s, startENF n) | s <-
      acceptstateENF n]

concatENFA :: ENFA Int -> ENFA Int -> ENFA Int -- Use only if states are disjoint again
concatENFA n1 n2 = ENFA states4 alphabet'' delta3 epT start3 accept where
  states4 = statesENF n1 ++ statesENF n2
  alphabet'' = alphabetENF n1 `union` alphabetENF n2
  delta3 sym st = deltaENF n1 sym st ++ deltaENF n2 sym st
  epT = epTrans n1 ++ epTrans n2 ++ [(s, startENF n2) | s <- acceptstateENF n1]
  start3 = startENF n1
  accept = acceptstateENF n2
```

Finally, we turn to implement DFA to RegExp. AFL-notes version:

First we rename the states so that they are now integers $1 \dots n$ where n is the amount of states. For that we use the following function:

```
makeIntDFA :: (Eq a, Ord a) => DFA a -> DFA Int
makeIntDFA dfa = DFA sts alph delt strt acceptst
  where sts = [1..length (states dfa)]
        alph = alphabet dfa
        delt sym i  = indX $ delta dfa sym (states dfa !! (i-1))
        strt = indX (start dfa)
        acceptst = [indX s | s <- acceptstate dfa]
        indX s = fromJust (elemIndex s (states dfa)) + 1

-- simplify to make the result a bit more readable
dfaToRegExp :: Ord a => DFA a -> RegExp
dfaToRegExp dfa = simplify $ regExpUnion [rijk dfaInt (start dfaInt) f (length $ states
    dfaInt) | f <- acceptstate dfaInt ]
  where dfaInt = (makeIntDFA . minimizeDFA) dfa

-- Here is the magic from the notes:
rijk :: DFA Int -> Int -> Int -> Int -> RegExp
rijk dfa i j 0 | i == j  = regExpUnion $ Epsilon : labels
               | null labels = Empty
               | length labels == 1 = head labels
               | otherwise = foldr Union (head labels) (tail labels)
  where labels =  [R [x] | x <- alphabet dfa, delta dfa x i == j]
rijk dfa i j k = Union (rijk dfa i j (k-1)) (Con (rijk dfa i k (k-1)) (Con (Star $ rijk dfa
     k k (k-1)) (rijk dfa k j (k-1))))
```

Brzozowski's algorithm for minimising DFA's

```
minimizeDFA :: Ord a => DFA a -> DFA [Int]
minimizeDFA df | all (`elem` acceptstate df) (reachables df)  = DFA [[0]] (alphabet df) (
    const (const [0])) [0] [[0]]
               |  otherwise = (reverseDFA . reverseDFA) df


reverseDFA :: Ord a => DFA a -> DFA [Int]
reverseDFA d = enfaToDFA' $ ENFA sts alph delt ep st acc where
    e = singleAccept $ cutDFA d
    sts = statesENF e
```

```
    alph = alphabetENF e
    delt sym s = [t | t <- statesENF e, s `elem` deltaENF e sym t]
    ep = [(s, t) | (t,s) <- epTrans e ]
    st = head $ acceptstateENF e
    acc = [startENF e]
    enfaToDFA' nf@(ENFA sts' alph' del eps strt ac) =
      cutDFA $ DFA sortedStates alph' del' (rtClose nf (sort [ t | (s, t) <- eps, s == strt
          ])) [st' | st' <- sortedStates,  intersect st' ac /= []] where
      del' sy ls = rtClose nf ( unionL [del sy l | l <- ls] )
      sortedStates = map sort $ subsequences sts'

singleAccept :: Ord a => DFA a -> ENFA Int
singleAccept d = ENFA sts alph delt ep st acc where
    d' = makeIntDFA d
    newAccept = maximum (states d') + 1
    sts = newAccept : states d'
    alph = alphabet d'
    delt sym t | t == newAccept = []
               | otherwise =  [delta d' sym t]
    ep = [(f, newAccept) | f <- acceptstate d']
    st = start d'
    acc = [newAccept]

-- Checks whether the reachable states of two DFA's are Isomorphic
brzozowski :: (Ord a, Ord b) => DFA a -> DFA b -> Bool
brzozowski d d' = alphabet d == alphabet d' &&
  (null (acceptstate m) && null (acceptstate m')) ||
   isIsomorphTo (start m) (start m') [] where
    m = minimizeDFA d
    m' = minimizeDFA d'
    isIsomorphTo s s' is =
      (s,s') `elem` is || (s `elem` acceptstate m) == (s' `elem` acceptstate m') &&
      all (\sym -> isIsomorphTo (delta m sym s) (delta m' sym s') ((s,s'):is)) (alphabet m)
```

8 Simple Tests

We create some tests, using QuickCheck to test whether some automata are equal, whether some regular expression generates exactly the language of some automaton, and we will add tests that show that our definitions work.

```
{-# LANGUAGE ScopedTypeVariables #-}
module Main where
import Test.QuickCheck
import RegExp
import DFA
import ENFA
import NFA
import Translation
--import DFA_raw

main :: IO()
main = do
    print "ZeroStart accepts words starting with 0:"
    test1DF
    print "ZeroStart does not accept other words:"
    test2DF
    print "ZeroStartNF accepts words starting with 0: "
    test1NF
    print "ZeroStartNF does not accept other words:"
    test2NF
    print "ZeroStartENF accepts words starting with 0:"
    test1ENF
    print "ZeroStartENF does not accept other words:"
    test2ENF
    print "Powerset construction yields equivalent DFA for NFA:"
    test1PowNF
```

```
    print "Powerset construction yields equivalent DFA for ENFA:"
    test1PowENF
    print "DFA accepts words generated by the corresponding RegExp:"
    test1DFtoReg
    print "DFA does not accept words generated by the RegExp corresponding to the
        complement DFA:"
    test2DFtoReg
    print "ENFA corresponding to RegExp accepts words generated from RegExp:"
    test1RegtoENF
    print "Composition of transitions yields equivalent DFA for DFA:"
    test1DFeqv
    print "Complement RegExps don't generate the same word:"
    test1CompRegx
    print "Complement on different RegExp:"
    test2CompRegx
    print "The reverse of the reverse accepts and rejects the same words:"
    test1ReverseDFA
    print "The reverse of the reverse is isomorphic to the original"
    test2ReverseDFA
    print "Minimize yields an automaton that accepts and rejects the same words:"
    test1MinimizeDFA
    print "Minimised automata generated from a RegExp accept words from said RegExp"
    testMinimizeMore
    print "Minimize doesn't yield more states"
    test2MinimizeDFA
    print "cutDFA dfa accepts and rejects the same as dfa"
    test1CutDFA
<<<<<<< HEAD
=======


>>>>>>> 7a82919 (Transition -> Translation)
```

The first test whether the example DFA, called zeroStart indeed accepts the strings starging with 0. Note the regular expression for this language is $0(0+1)*$. We do the same for slightly modified NFA and ENFA versions of zeroStart.

```
zeroStartNF:: NFA Int
zeroStartNF = NFA [0,1,2, 3] ['0', '1'] deltazeroNF 0 [1] where
    deltazeroNF:: Symbol-> Int -> [Int]
    deltazeroNF char st
     | st == 0 && char == '0' = [1]
     | st == 0 && char == '1' = [2, 3]
     | st == 1 = [1]
     | st == 2 = [2, 3]
     | st == 3 = [3]
     | otherwise = [-1]

zeroStartENF:: ENFA Int
zeroStartENF = ENFA [0,1,2, 3] ['0', '1'] deltazeroNF [(3,2), (2,3)] 0 [1] where
    deltazeroNF:: Symbol-> Int -> [Int]
    deltazeroNF char st
     | st == 0 && char == '0' = [1]
     | st == 0 && char == '1' = [2, 3]
     | st == 1 = [1]
     | st == 2 = [2, 3]
     | st == 3 = [3]
     | otherwise = [-1]

dfa2:: DFA Int
dfa2 = DFA [0,1,3] "01" del2 3 [0] where
    del2 char st
      | char =='0' && st == 0 = 3
      | char =='0' && st == 1 = 0
      | char =='0' && st == 3 = 3
      | char =='1' && st == 0 = 1
      | char =='1' && st == 1 = 3
      | char =='1' && st == 3 = 3
      | otherwise = -1

-- test
```

```
test1DF :: IO ()
test1DF = quickCheck (forAll (generateString (Con (R "0") (Star (Union (R "0") (R "1")))))
                        (\w -> zeroStart `evaluate` w))

test1NF :: IO ()
test1NF = quickCheck (forAll (generateString (Con (R "0") (Star (Union (R "0") (R "1")))))
                        (\w -> zeroStartNF `evaluateNF` w ))

test1ENF :: IO ()
test1ENF = quickCheck (forAll (generateString (Con (R "0") (Star (Union (R "0") (R "1")))))
                        (\w -> zeroStartENF `evaluateENF` w ))
```

We now test that words generated from the RegExp for the complement are not accepted. Note the RegExp for the complement language is $\epsilon + 1(0+1)*$

```
test2DF :: IO ()
test2DF = quickCheck (forAll (generateString (Union Epsilon (Con (R "1") (Star (Union (R "0
    ") (R "1")))))))
                        (not . evaluate zeroStart))

test2NF :: IO ()
test2NF = quickCheck (forAll (generateString (Union Epsilon (Con (R "1") (Star (Union (R "0
    ") (R "1")))))))
                        (not . evaluateNF zeroStartNF))

test2ENF :: IO ()
test2ENF = quickCheck (forAll (generateString (Union Epsilon (Con (R "1") (Star (Union (R "
    0") (R "1")))))))
                        (not . evaluateENF zeroStartENF))
```

We now test the powerset construction. We test if an NFA accepts the same binary strings as its powerset construct. We repeat for ENFA-s. We generate arbitrary NFA-s and ENFA-s for this.

```
test1PowNF :: IO ()
test1PowNF = quickCheck (forAll (generateString (Union (R "0") (R "1")))
                            (\w (enf:: NFA Int) -> evaluateNF enf w == evaluate (nfaToDFA
                                enf) w) )

test1PowENF :: IO ()
test1PowENF = quickCheck (forAll (generateString (Union (R "0") (R "1")))
                            (\w (enf:: ENFA Int) -> evaluateENF enf w == evaluate (
                                enfaToDFA enf) w) )
```

We now test the transition from DFA to RegExp. We check whether a DFA accepts words generated from the corresponding RegExp and that it does not accept words generated from the RegExp corresponding to the complement DFA. We generate arbitrary DFA-s for this.

```
test1DFtoReg :: IO ()
test1DFtoReg = quickCheck (\ (df:: DFA Int) -> if dfaToRegExp df /= Empty then
                forAll (generateString (dfaToRegExp df)) (\w -> df `evaluate` w) else
                    property True)

test2DFtoReg :: IO ()
test2DFtoReg = quickCheck ( \ (df:: DFA Int) -> if dfaToRegExp (flipDFA df) /= Empty then
                forAll (generateString (dfaToRegExp (flipDFA df))) (not . evaluate df )
                    else property True)
```

We now test the transition from RegExp to ϵ-NFA. We check whether the ϵ-NFA corresponding to a RegExp accepts words generated from the RegExp. We generate arbitrary RegExps- for this.

```
test1RegtoENF :: IO ()
test1RegtoENF = quickCheck ( \ (reg:: RegExp) -> forAll (generateString reg)
                        (\w -> regExpToENFA reg `evaluateENF` w))
```

We put everything together and test whether a DFA and the DFA obtained from transitioning to RegExp, then to ϵ-NFA and back to DFA are equivalent. We test it on our two example DFA-s. We also calculate complement RegExp-s by flipping the corresponding DFA accept states and test for a for 2 RegExps that they don't generate the same words.

```
test1DFeqv :: IO ()
test1DFeqv = quickCheck (forAll (generateString (Union (R "0") (R "1")))
                (\w -> evaluate dfa2 w == evaluate (func dfa2) w &&  evaluate zeroStart w
                    == evaluate (func zeroStart) w)) where
                    func = enfaToDFA . regExpToENFA . dfaToRegExp



test1CompRegx :: IO ()
test1CompRegx = quickCheck (forAll (generateString reg) (\w -> forAll (generateString (creg
     reg))(/= w))) where
                    creg = dfaToRegExp . makeIntDFA . flipDFA . enfaToDFA . regExpToENFA
                    reg = Union Epsilon (Con (R "1") (Star (Union (R "0") (R "1"))))

test2CompRegx :: IO ()
test2CompRegx = quickCheck (forAll (generateString reg) (\w -> forAll (generateString (creg
     reg))(/= w))) where
                    creg = dfaToRegExp . makeIntDFA . flipDFA . enfaToDFA . regExpToENFA
                    reg = Con (Star (R "2")) (Con (R "0") (R "0"))



test1ReverseDFA :: IO ()
test1ReverseDFA = quickCheck (\r (d:: DFA Int)  -> forAll (generateString r) (\w -> d `
    evaluate` w == (reverseDFA . reverseDFA ) d `evaluate` w ))
test2ReverseDFA :: IO ()
test2ReverseDFA = quickCheck (\(d :: DFA Int) -> brzozowski d ((reverseDFA . reverseDFA) d)
     )
test1MinimizeDFA :: IO ()
test1MinimizeDFA = quickCheck (\r (d :: DFA Int) -> forAll (generateString r) (\w -> d `
    evaluate` w == minimizeDFA d `evaluate` w ))
test2MinimizeDFA :: IO ()
test2MinimizeDFA = quickCheck (\ (d :: DFA Int) -> length (states d) >= length (states (
    minimizeDFA d)))

testMinimizeMore :: IO ()
testMinimizeMore = quickCheck (\r -> forAll (generateString r) (\w -> (minimizeDFA .
    enfaToDFA . regExpToENFA) r `evaluate` w))


test1CutDFA :: IO ()
test1CutDFA = quickCheck (\r (d :: DFA Int) -> forAll (generateString r) (\w -> d `evaluate
    ` w == cutDFA d `evaluate` w ))
<<<<<<< HEAD
=======

>>>>>>> 7a82919 (Transition -> Translation)
```

9 Conclusion

Our primary sources of reference are [Chi18] and [Sip13].

References

[Chi18] Maurice Chiodo. Automata and formal languages lecture notes, December 2018.

[Jel14] Tikhon Jelvis. Transitive closure from a list using haskell. Computer Science Stack Exchange, 2014. [Online:] `https://stackoverflow.com/questions/19212558`.

[Sip13] Michael Sipser. *Introduction to the Theory of Computation*. Course Technology, Boston, MA, third edition, 2013.